INFORME FINAL

Proyecto Integrador Base de Datos

Universidad Tecnológica Nacional, Facultad Regional San Rafael
(UTN FRSR)

Integrantes: Agustín Chaumont, Ezequiel Mautino, Pablo Burgos y Genaro De Boni

ÍNDICE

1. Introducción y alcance **2**

2. Modelo de datos **3**

2.1. Diagrama Entidad Relación 3

2.2. Esquema Relacional 3

2.3. Diccionario de Datos 4

• Tipos de datos propios en PosgreSQL (DOMINIO y ENUM) 4

• Personal y Locaciones 4

• Catálogos Fijos 5

• Producción Agrícola (Uvas) 6

• El Producto (Vinos) 7

• Jerarquía de Clientes 7

• Movimientos y Stock 8

2.4. Decisiones de Diseño 10

3. Integridad y lógica en la base **11**

3.1. Tabla de Trazabilidad 11

3.2. Demostración de concurrencia 12

4. Carga masiva **14**

4.1. Catálogos 14

4.2. Orden de carga 14

4.3. Estrategia frente a los triggers 14

4.4. Tiempos y consultas de control 15

5. Índices y rendimiento **17**

5.1. Tabla comparativa antes y después 17

5.2. Planes de ejecución 17

5.3. Costo de escritura 18

6. Consultas SQL **19**

7. API y front **20**

7.1. Arquitectura 20

7.2. Endpoints 20

7.3. Traducción de errores 20

7.4. Experiencias con el Frontend 21

8. Conclusiones **22**

1. Introducción y alcance:

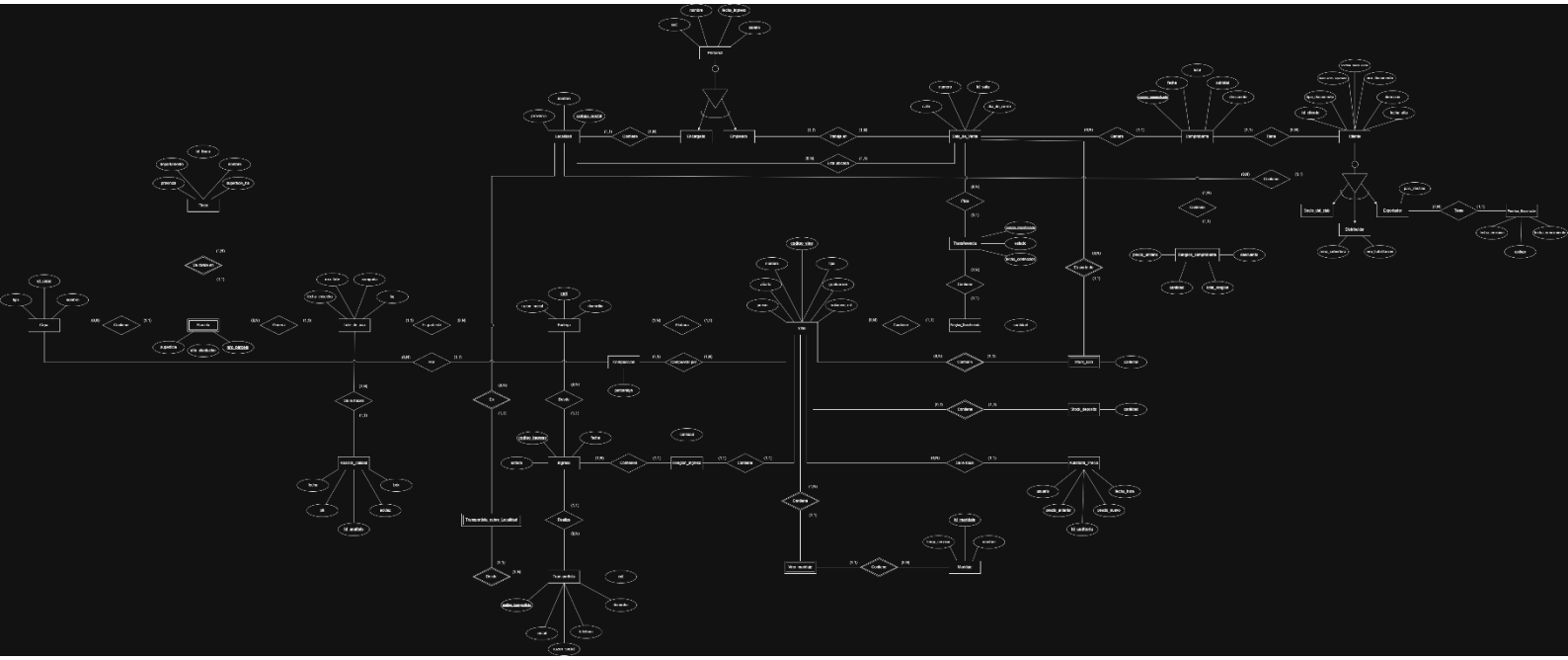
Este informe documenta el desarrollo del Trabajo Práctico Integrador correspondiente a la asignatura Bases de Datos de la carrera Ingeniería en Sistemas (UTN Facultad Regional San Rafael). El proyecto consiste en el diseño, implementación y puesta en producción del sistema de gestión y distribución para «Bodegas del Atuel», que integra tanto la producción de materia prima como la elaboración y comercialización de sus productos.

El alcance del negocio contempla el ciclo completo de la empresa, lo dividimos en 4 partes:

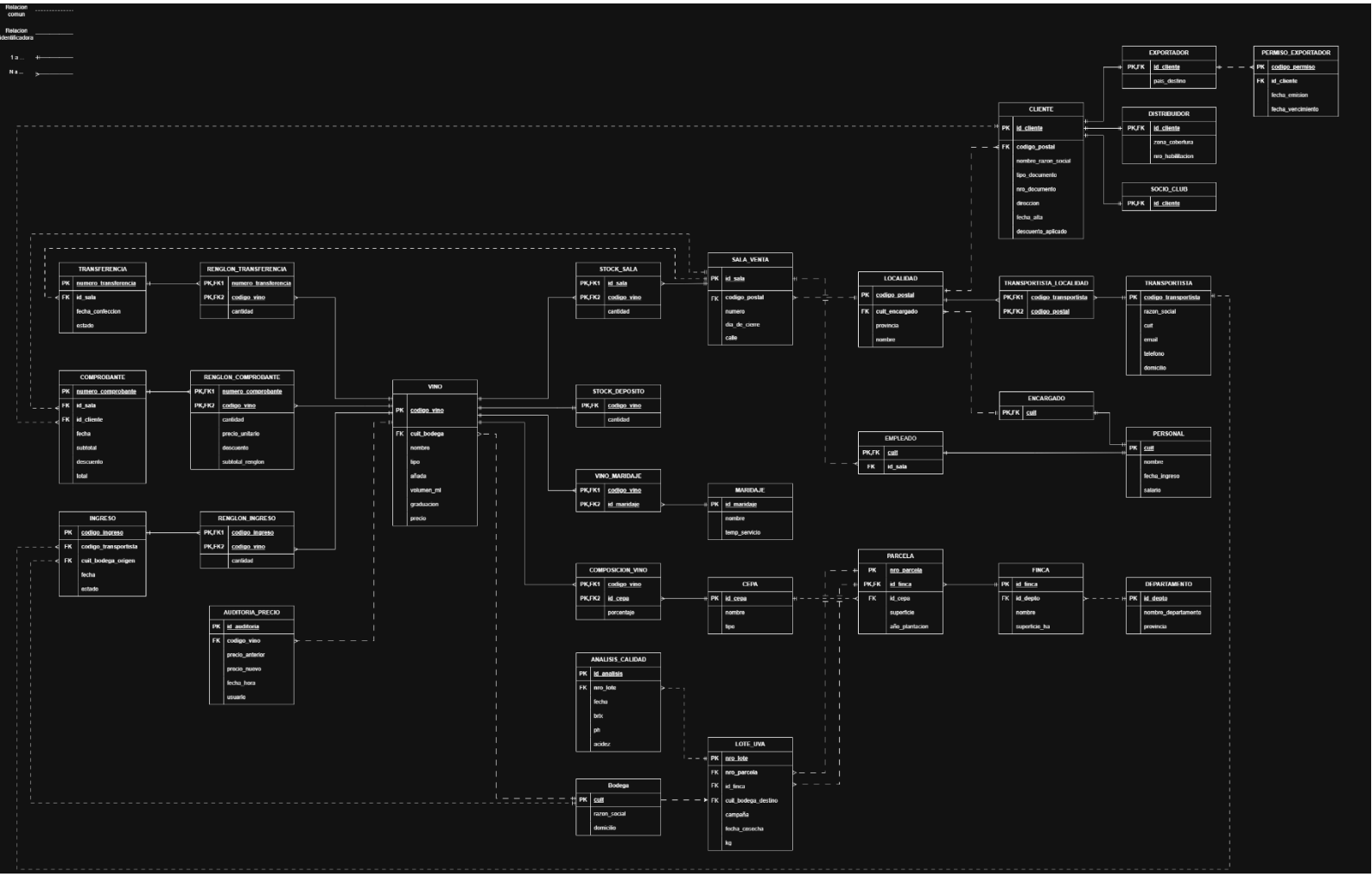
- **Producción Agrícola:** Gestión de fincas, parcelas, cepas y el registro de lotes de uva cosechados por campaña, incluyendo sus respectivos análisis de calidad.
- **Elaboración y Catálogo:** Administración de las bodegas del grupo, los vinos elaborados, la composición de cepas y sugerencias de maridaje.
- **Logística y Stock:** Control de inventario en un depósito central y en salas de venta distribuidas regionalmente, gestionando el flujo de mercadería a través de ingresos (desde bodegas) y transferencias (hacia las salas) mediante empresas de transporte.
- **Comercialización:** Registro de clientes categorizados en una jerarquía (socios del club, distribuidores y exportadores) con beneficios específicos, y la emisión de comprobantes de venta con validación estricta de stock disponible y cálculo automático de descuentos.

2. Modelo de datos:

2.1. Diagrama Entidad Relación:



2.2. Esquema Relacional:



2.3. Diccionario de Datos:

2.3.1. Tipos de datos propios en PosgreSQL (DOMINIO y ENUM)

`cuit_dom` (DOMINIO): Basado en `CHAR(11)`. Restricción: `CHECK (VALUE ~ '^[0-9]{11}$')`. Valida que el CUIT contenga exactamente 11 dígitos numéricos sin guiones.

`estado_documento` (ENUM): Valores permitidos: 'en_confeccion', 'terminado', 'procesado'. Utilizado en ingresos y transferencias.

`tipo_vino` (ENUM): Valores permitidos: 'tinto', 'blanco', 'rosado', 'espumante'.

`tipo_cepa` (ENUM): Valores permitidos: 'tinta', 'blanca'.

2.3.2. Personal y Locaciones

Tabla: PERSONAL

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
cuit	cuit_dom	NO	PK	Identificador único del personal (CUIT).
nombre	VARCHAR(100)	NO		Nombre completo del empleado/encargado.
fecha_ingreso	DATE	NO		Fecha en la que ingresó a la empresa.
salario	NUMERIC(10,2)	NO	CHECK (> 0)	Salario bruto. Nunca usar FLOAT.

Tabla: ENCARGADO

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
cuit	cuit_dom	NO	PK, FK -> PERSONAL	CUIT del encargado regional.

Tabla: EMPLEADO

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
cuit	cuit_dom	NO	PK, FK -> PERSONAL	CUIT del empleado de sala.
id_sala	INTEGER	NO	FK -> SALA_VENTA	Sala en la que trabaja actualmente.

Tabla: LOCALIDAD

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo_postal	VARCHAR(10)	NO	PK	Código postal de la localidad.
provincia	VARCHAR(50)	NO		Nombre de la provincia.
Nombre	VARCHAR(100)	NO		Nombre de la ciudad o localidad.
cuit_encargado	cuit_dom	NO	FK -> ENCARGADO	Encargado regional asignado (Uno por loc.).

Tabla: SALA_VENTA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_sala	SERIAL	NO	PK	Identificador autogenerado de la sala.
numero	INTEGER	NO		Número de la calle de la sucursal.
Calle	VARCHAR(100)	NO		Nombre de la calle.
dia_de_cierre	SMALLINT	SÍ	CHECK (1 a 7)	Día de descanso (1=Lunes, 7=Domingo).
codigo_postal	VARCHAR(10)	NO	FK -> LOCALIDAD	Localidad donde se ubica.

2.3.3. Catálogos Fijos

Tabla: BODEGA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
cuit	cuit_dom	NO	PK	CUIT de la bodega.
razon_social	VARCHAR(100)	NO		Razón social legal.
domicilio	VARCHAR(150)	NO		Domicilio completo.

Tabla: CEPA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_cepa	SERIAL	NO	PK	Identificador interno de la cepa.
nombre	VARCHAR(50)	NO	UNIQUE	Nombre (ej: Malbec, Syrah).
tipo	tipo_cepa	NO		Clasificación (Tinta o Blanca).

Tabla: MARIDAJE

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_maridaje	SERIAL	NO	PK	ID autogenerado.
nombre	VARCHAR(100)	NO		Ej: 'Carnes rojas', 'Quesos duros'.
temp_servicio	NUMERIC(4,2)	NO		Temperatura recomendada en grados Celsius.

Tabla: TRANSPORTISTA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo	SERIAL	NO	PK	Código interno del transportista.
razon_social	VARCHAR(100)	NO		Nombre de la empresa de transporte.
cuit	cuit_dom	NO	UNIQUE	CUIT de facturación.
email	VARCHAR(100)	SÍ		Correo de contacto.
telefono	VARCHAR(20)	SÍ		Teléfono de contacto.
domicilio	VARCHAR(150)	SÍ		Dirección física.

Tabla: TRANSPORTISTA_LOCALIDAD (N:M)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo_transp.	INTEGER	NO	PK, FK -> TRANSPORT.	Empresa de transporte.
codigo_postal	VARCHAR(10)	NO	PK, FK -> LOCALIDAD	Localidad a la que tiene alcance/cobertura.

2.3.4. Producción Agrícola (Uvas)

Tabla: DEPARTAMENTO

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_depto	SERIAL	NO	PK	ID interno.
nombre_depto	VARCHAR(100)	NO		Nombre (Ej: San Rafael, Luján de Cuyo).
provincia	VARCHAR(50)	NO		Provincia del departamento.

Tabla: FINCA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_finca	SERIAL	NO	PK	ID autogenerado de la finca.
nombre	VARCHAR(100)	NO		Nombre comercial de la finca.
superficie_ha	NUMERIC(10,2)	NO	CHECK (> 0)	Hectáreas totales.
id_depto	INTEGER	NO	FK -> DEPARTAMENTO	Ubicación geográfica normalizada.

Tabla: PARCELA (Entidad Débil)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
nro_parcela	INTEGER	NO	PK	Número identificador dentro de la finca.
id_finca	INTEGER	NO	PK, FK -> FINCA	Finca a la que pertenece.
superficie	NUMERIC(10,2)	NO	CHECK (> 0)	Hectáreas específicas de la parcela.
año_plantacion	INTEGER	NO	CHECK (>= 1800)	Año de siembra del viñedo.
id_cepa	INTEGER	NO	FK -> CEPA	Variedad plantada.

Tabla: LOTE_UVA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
nro_lote	SERIAL	NO	PK	ID autogenerado de la cosecha.
campaña	INTEGER	NO	CHECK (> 0)	Año de la vendimia.
fecha_cosecha	DATE	NO		Fecha exacta de la recolección.
Kg	NUMERIC(10,2)	NO	CHECK (> 0)	Rendimiento bruto de uva.
nro_parcela	INTEGER	NO	FK -> PARCELA	(Con id_finca) Parcela cosechada. UNIQUE*.
id_finca	INTEGER	NO	FK -> PARCELA	(Con nro_parcela). UNIQUE*.
cuit_bodega	cuit_dom	NO	FK -> BODEGA	Bodega de destino para elaborar el vino.

Tabla: ANALISIS_CALIDAD

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_analisis	SERIAL	NO	PK	Código del análisis.
fecha	TIMESTAMP	NO		Momento del testeo.
brix	NUMERIC(5,2)	NO	CHECK (> 0)	Grados Brix (azúcar).
ph	NUMERIC(4,2)	NO	CHECK (> 0)	Potencial de Hidrógeno.
acidez	NUMERIC(5,2)	NO	CHECK (> 0)	Acidez total en g/L.
nro_lote	INTEGER	NO	FK -> LOTE_UVA	Lote analizado.

2.3.5. El Producto (Vinos)

Tabla: VINO

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo_vino	SERIAL	NO	PK	Identificador comercial.
nombre	VARCHAR(100)	NO	UNIQUE (Compuesta)*	Nombre del vino (Ej: Malbec Reserva).
tipo	tipo_vino	NO		Tinto, blanco, rosado o espumante.
añada	INTEGER	NO	UNIQUE (Compuesta)*	Año de elaboración.
volumen_ml	INTEGER	NO	UNIQUE (Compuesta)*	Ej: 750, 1500.
graduacion	NUMERIC(4,2)	NO	CHECK (BETWEEN 5 AND 20)	Alcohol (RN-15).
precio	NUMERIC(10,2)	NO	CHECK (> 0)	Precio unitario universal (RN-01).
cuit_bodega	cuit_dom	NO	FK -> BODEGA	Bodega productora.

Tabla: COMPOSICION_VINO (N:M)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo_vino	INTEGER	NO	PK, FK -> VINO	Vino compuesto.
id_cepa	INTEGER	NO	PK, FK -> CEPA	Cepa participante en el corte.
porcentaje	NUMERIC(5,2)	NO	CHECK (> 0 Y <= 100)	Peso relativo. Un trigger valida sum(100).

Tabla: VINO_MARIDAJE (N:M)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo_vino	INTEGER	NO	PK, FK -> VINO	
id_maridaje	INTEGER	NO	PK, FK -> MARIDAJE	

2.3.6. Jerarquía de Clientes

Tabla: CLIENTE (Superclase)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_cliente	SERIAL	NO	PK	Identificador único de sistema.
nombre_razon	VARCHAR(150)	NO		Apellido y Nombre o Razón Social.
tipo_doc	VARCHAR(10)	NO		Ej: DNI, CUIT.
nro_doc	VARCHAR(20)	NO	UNIQUE	Número de identificación.
direccion	VARCHAR(150)	SÍ		Calle y número.
fecha_alta	DATE	NO	DEFAULT CURRENT_DATE	Fecha de registro.
descuento	NUMERIC(5,2)	NO	CHECK (>= 0)	Porcentaje de bonificación base.
codigo_postal	VARCHAR(10)	NO	FK -> LOCALIDAD	Ciudad de residencia/operación.

Tabla: SOCIO_CLUB (Subclase)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_cliente	INTEGER	NO	PK, FK -> CLIENTE	Vincula 1:1 con el cliente base.

Tabla: DISTRIBUIDOR (Subclase)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_cliente	INTEGER	NO	PK, FK -> CLIENTE	Vincula 1:1 con el cliente base.
zona_cobertura	VARCHAR(100)	NO		Territorio de distribución asignado.
nro_habilitacion	VARCHAR(50)	NO	UNIQUE	Permiso comercial / RENSPA.

Tabla: EXPORTADOR (Subclase)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_cliente	INTEGER	NO	PK, FK -> CLIENTE	Vincula 1:1 con el cliente base.
pais_destino	VARCHAR(50)	NO		Mercado principal exterior.

Tabla: PERMISO_EXPORTADOR

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo_permiso	VARCHAR(50)	NO	PK	Identificador del certificado emitido.
fecha_emision	DATE	NO		
fecha_venc	DATE	NO	CHECK (venc > emision)	Valida que expire después de la emisión.
id_cliente	INTEGER	NO	FK -> EXPORTADOR	Exportador dueño del permiso.

2.3.7. Movimientos y Stock

Tabla: STOCK_DEPOSITO

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
codigo_vino	INTEGER	NO	PK, FK -> VINO	Identifica al vino almacenado.
cantidad	INTEGER	NO	CHECK (>= 0) DEFAULT 0	Botellas totales en el centro logístico.

Tabla: STOCK_SALA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_sala	INTEGER	NO	PK, FK -> SALA_VENTA	Sucursal de venta al público.
codigo_vino	INTEGER	NO	PK, FK -> VINO	Identifica al vino almacenado.
cantidad	INTEGER	NO	CHECK (>= 0) DEFAULT 0	Inventario de la sucursal.

Tabla: COMPROBANTE (Ventas)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
nro_comprob.	SERIAL	NO	PK	Autonumérico de facturación.
fecha	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Fecha y hora de venta.
subtotal	NUMERIC(12,2)	NO	DEFAULT 0	Sumatoria sin descuentos.
descuento	NUMERIC(12,2)	NO	DEFAULT 0	Monto descontado.
total	NUMERIC(12,2)	NO	DEFAULT 0	Importe final a pagar.
id_cliente	INTEGER	NO	FK -> CLIENTE	Comprador.
id_sala	INTEGER	NO	FK -> SALA_VENTA	Sucursal que facturó.

Tabla: RENGLON_COMPROBANTE

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
nro_comprob.	INTEGER	NO	PK, FK -> COMPROBANTE	
codigo_vino	INTEGER	NO	PK, FK -> VINO	Producto vendido.
cantidad	INTEGER	NO	CHECK (> 0)	Unidades llevadas.
precio_unitario	NUMERIC(10,2)	NO	CHECK (> 0)	Precio congelado al momento de la venta.
descuento	NUMERIC(10,2)	NO	CHECK (>= 0)	Descuento aplicado a este producto.
subtotal	NUMERIC(12,2)	NO	CHECK (> 0)	(precio_unitario * cant) - descuento.

Tablas: INGRESO y TRANSFERENCIA

(Comparten estructura base y comportamiento)

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_documento	SERIAL	NO	PK	Número de Ingreso o Transferencia.
fecha	DATE	NO	DEFAULT CURRENT_DATE	Momento de confección.
estado	estado_doc	NO	DEFAULT 'en_confeccion'	Transición manejada por triggers/SP.
fk_origen_dest	INTEGER	NO	FK -> BODEGA/SALA	Dependiendo del tipo de doc.

Tablas: RENGLON_INGRESO / RENGLON_TRANSFERENCIA

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_documento	INTEGER	NO	PK, FK -> Cabecera	Cabecera del documento.
codigo_vino	INTEGER	NO	PK, FK -> VINO	Producto transportado.
cantidad	INTEGER	NO	CHECK (> 0)	Unidades a mover (botellas).

Tabla: AUDITORIA_PRECIO

Columna	Tipo de Dato	Nulo	Restricciones	Descripción
id_auditoria	SERIAL	NO	PK	ID de registro.
precio_anterior	NUMERIC(10,2)	NO		Valor antes del UPDATE.
precio_nuevo	NUMERIC(10,2)	NO		Valor establecido tras el UPDATE.
fecha_hora	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Momento exacto del cambio.
usuario	VARCHAR(50)	NO	DEFAULT CURRENT_USER	Usuario de BD que ejecutó el cambio.
codigo_vino	INTEGER	NO	FK -> VINO	Vino afectado.

2.4. Decisiones de Diseño:

Consideramos que no hubieron grandes decisiones de diseño ya que la gran mayoría del trabajo viene especificado y explicado en el trabajo practico el cómo hay que hacerlas.

Diría que la mas considerable es haber definido la superclase PERSONAL, con las subclases ENCARGADO y EMPLEADO. Esto lo hicimos porque si manteníamos ambos diferenciados únicamente con un atributo (como por ejemplo tipo o puesto) entonces tendríamos menos integridad referencial y en ocasiones valores nulos.

3. Integridad y lógica en la base:

3.1. Tabla de Trazabilidad:

N°	Descripción de la Regla	Mecanismo Implementado en BD	Caso de Prueba SQL (tests/db_tests.sql)
RN-01	El precio es único y mayor que cero.	Atributo precio en VINO + CHECK (precio > 0).	**Éxito:** INSERT INTO VINO con precio $12500.00. **Fallo esperado:** INSERT INTO VINO (ID 9099) con precio $0.00.
RN-02	Sala en una sola localidad. Único encargado por localidad.	Normalización (3FN) con FOREIGN KEY.	**Éxito:** Inserción de salas 901 y 902 en CP 5600. **Fallo esperado:** INSERT INTO SALA_VENTA con CP inexistente (9999) y LOCALIDAD con CUIT encargado falso.
RN-03	Cada empleado trabaja en una sola sala.	id_sala en EMPLEADO como FK + NOT NULL.	**Fallo esperado 1:** Reasignar empleado a sala 902 (Error UNIQUE). **Fallo esperado 2:** INSERT de empleado con id_sala NULL.
RN-04	Una cepa por parcela, cosechada máximo una vez por campaña.	UNIQUE (campaña, nro_parcela, id_finca) en LOTE_UVA.	**Éxito:** Cosecha campaña 2025 de parcela 1. **Fallo esperado:** Segundo INSERT de cosecha para parcela 1 en campaña 2025.
RN-05	Porcentajes de corte suman exactamente 100.	Constraint trigger diferido (DEFERRABLE INITIALLY DEFERRED) sobre tabla COMPOSICION_VINO .	**Éxito:** INSERT sumando 100% y luego COMMIT. **Fallo esperado:** INSERT de vino 9003 sumando 80%; aborta transacción al hacer COMMIT.
RN-06	Descuento: socio 10%, distribuidor 20%, exportador 30%.	Función + CHECK de rango de descuento.	**Éxito:** INSERT de 3 clientes válidos (9001 a 9003). **Fallo esperado:** INSERT de cliente (ID 9099) con descuento ilegal del 75.00%.
RN-07	Vencimiento del permiso posterior a la emisión.	CHECK (fecha_venc > fecha_emision).	**Éxito:** INSERT de 2 permisos vigentes. **Fallo esperado:** INSERT con fecha_venc ('2026-02-01') anterior a fecha_emision ('2026-05-01').
RN-08	El total es la suma de los renglones (No manual).	Función de cálculo + Trigger AFTER INSERT.	**Éxito:** INSERT de comprobante cabecera en $0.00 y renglón con importes falsos ($1.00). Un SELECT demuestra el recálculo correcto a $22500.00.
RN-09	Stock nunca negativo y no se puede vender sin stock.	Trigger BEFORE INSERT + CHECK (cantidad >= 0).	**Fallo esperado 1:** INSERT de renglón pidiendo 5 botellas del vino 9002 (Stock = 0). **Fallo esperado 2:** UPDATE STOCK_SALA SET cantidad = -5.

RN-10	Ingreso terminado impacta depósito. Estados sin retroceso.	ENUM estado_documento + trigger + procedimiento sp_procesar_ingreso.	**Fallo esperado 1:** UPDATE de ingreso 9001 de estado procesado a terminado. **Fallo esperado 2:** CALL sp_procesar_ingreso(9002) estando aún en_confeccion.
RN-11	Fecha actual inmodificable y control de transferencia.	Trigger de fecha/estado + procedimiento sp_procesar_transferencia.	**Fallo esperado 1:** UPDATE de la fecha de la transferencia 9001. **Fallo esperado 2:** CALL sp_procesar_transferencia(9002) sin stock de vino 9002 en depósito (Fallo atómico).
RN-12	Modificar renglón solo admitido en estado 'en_confeccion'.	Trigger BEFORE INSERT/UPDATE/DELETE en renglones.	**Fallo esperado 1:** INSERT de renglón en ingreso 9001 (estado procesado). **Fallo esperado 2:** DELETE de renglón en transferencia 9002 (estado terminado).
RN-13	Todo cambio de precio queda auditado.	Trigger AFTER UPDATE sobre VINO + tabla AUDITORIA_PRECIO.	**Éxito:** UPDATE VINO SET precio = 14200.00 para ID 9001 y SELECT de validación. **Fallo esperado:** INSERT directo en AUDITORIA_PRECIO sobre vino 99999 (Inexistente).
RN-14	No borrar transportista, bodega ni vino con movimientos.	Claves foráneas con ON DELETE RESTRICT.	**Fallo esperado:** 3 sentencias DELETE FROM intentando borrar Transportista 900, Bodega 30700000018 y Vino 9001.
RN-15	Cantidades, kilos, superficies positivas. Graduación 5-20.	Restricciones CHECK (> 0 y BETWEEN).	**Fallos esperados:** INSERT de vino con 24.5° de graduación, Finca con superficie -15.00 y Lote de uva con 0.00 Kg cosechados.

3.2. Demostración de concurrencia:

El escenario consistió en preparar un estado inicial donde la sala de venta 901 contaba con un stock de exactamente 1 botella del vino 9001. Luego, dos sesiones independientes (psql A y B) intentaron vender esa única botella simultáneamente.

Escenario 1: Con bloqueo explícito (SELECT ... FOR UPDATE) Se utilizaron los scripts concurrencia_sesion_A_for_update.sql y concurrencia_sesion_B_for_update.sql.

1. **Sesión A** inició la transacción y ejecutó SELECT cantidad ... FOR UPDATE; obteniendo 1 botella y bloqueando la fila inmediatamente.
2. **Sesión B** intentó realizar la misma consulta y quedó en estado de espera (bloqueada) por 3 segundos, ya que la Sesión A mantenía el cerrojo.
3. **Sesión A** descontó el stock a 0 e insertó el renglón. Al momento de confirmar, el Trigger asociado a la RN-09 ("Stock insuficiente") rechazó la operación mediante un RAISE EXCEPTION y ejecutó un ROLLBACK, ya que el stock disponible pasó a ser 0 y se solicitaba 1 botella.
4. Al finalizar la transacción A, **Sesión B** se desbloqueó, leyó el stock actualizado e intentó realizar un UPDATE a -1. Esta acción falló y fue abortada gracias a la restricción CHECK (cantidad >= 0) implementada como chk_stock_sala_no_negativo.

Escenario 2: Sin bloqueo explícito (ausencia de FOR UPDATE) Se restauró el stock a 1 y se utilizaron los scripts concurrencia_sesion_A_no_lock.sql y concurrencia_sesion_B_no_lock.sql.

1. Ambas sesiones ejecutaron un SELECT simple en simultáneo, y debido al aislamiento READ COMMITTED, ambas leyeron que la cantidad disponible era 1. Se produjo una condición de carrera.
2. **Sesión A** intentó registrar la venta y falló por el Trigger RN-09 ("Stock insuficiente").
3. **Sesión B** intentó realizar el UPDATE restando una botella (a -1) basándose en su lectura inicial, fallando por la restricción CHECK (cantidad >= 0). *Análisis:* Aunque la base de datos protegió la integridad de los datos en ambos casos (el stock nunca bajó de cero gracias al CHECK y al Trigger), la ausencia de FOR UPDATE permitió que ambas sesiones operaran sobre la ilusión del mismo stock inicial, generando procesamiento innecesario de transacciones destinadas a fallar.

4. Carga masiva:

4.1. Catálogos:

La migración de datos se inició poblando las tablas maestras o de referencia del sistema (CEPA, MARIDAJE, DEPARTAMENTO, LOCALIDAD, BODEGA y TRANSPORTISTA). Esta estrategia se adoptó para respetar estrictamente la jerarquía de dependencias del esquema relacional: al poblar primero estas entidades fuertes, se garantizó la disponibilidad de las claves primarias (PK) necesarias para validar las Foreign Keys (FK) de las entidades transaccionales y de producción.

4.2. Orden de carga:

La secuencia se organizó en los siguientes niveles jerárquicos:

1. **Recursos Humanos:** Primero se cargó la tabla PERSONAL y sus subtipos (ENCARGADO y EMPLEADO), ya que entidades geográficas y organizacionales requieren asociar a un responsable.
2. **Locaciones y Catálogos Restantes:** Se cargaron DEPARTAMENTO, LOCALIDAD (que consume el CUIT del encargado), BODEGA y TRANSPORTISTA.
3. **Estructura Comercial y Producción:** Por el lado comercial se insertaron las SALA_VENTA. Por el lado agrícola, se respetó la jerarquía de dependencia física: FINCA primero, luego sus PARCELAs, seguido de LOTE_UVA y ANALISIS_CALIDAD.
4. **Productos y Stock:** Se dio de alta el catálogo principal (VINO) junto con sus tablas de relación (COMPOSICION vinculando con CEPA, y VINO_MARIDAJE vinculando con MARIDAJE). Luego se establecieron los saldos iniciales en STOCK_DEPOSITO y STOCK_SALA.
5. **Transacciones y Operaciones:** Con toda la estructura de soporte ya poblada, se insertó la base de CLIENTEs, seguida por los encabezados de los documentos (INGRESO, TRANSFERENCIA y COMPROBANTE).
6. **Detalles Masivos:** El proceso finalizó con las tablas de mayor volumen, como los renglones de ingresos y RENGLON_COMPROBANTE, que cierran la cadena de dependencias al requerir tanto el encabezado del documento como el vino en cuestión.

4.3. Estrategia frente a los triggers:

Para la carga masiva de las tablas, la ejecución de los triggers por cada fila insertada representaba un cuello de botella muy grande. Si el motor evaluaba las reglas de negocio registro por registro durante el comando COPY, el tiempo de procesamiento se disparaba.

La estrategia adoptada consistió en deshabilitar temporalmente los triggers en las tablas más pesadas. La secuencia de ejecución fue la siguiente:

1. Se suspendieron los triggers utilizando el comando ALTER TABLE tabla DISABLE TRIGGER ALL;.

2. Se ejecutó la inserción masiva leyendo directamente desde los CSV.
3. Se restauró el comportamiento normal con ALTER TABLE tabla ENABLE TRIGGER ALL;.

4.4. Tiempos y consultas de control:

Para garantizar la viabilidad de la carga masiva, se analizó el rendimiento de diferentes estrategias de inserción. Se comprobó mediante un benchmark que una inserción fila por fila procesaba 5.000 filas en 0,635 segundos (aproximadamente 7.874 filas/segundo), lo que hubiese implicado un estimado de 63,5 segundos para cargar 500.000 registros. En contraste, la utilización de cargas por lotes mediante COPY redujo drásticamente el tiempo a 0,008 segundos para 5.000 filas (alrededor de 625.000 filas/segundo), estimando apenas 0,8 segundos para el medio millón de registros.

Tabla	Filas	Tiempo (s)
departamento	18	0.007
personal	78	0.004
encargado	18	0.003
localidad	18	0.004
sala_venta	30	0.003
empleado	60	0.004
cepa	12	0.002
maridaje	8	0.002
bodega	20	0.003
transportista	15	0.003
transportista_localidad	40	0.003
finca	40	0.003
parcela	600	0.013
vino	400	0.010
composicion_vino	802	0.018
vino_maridaje	806	0.021
cliente	20000	0.282
socio_club	12000	0.110
distribuidor	5000	0.073
exportador	3000	0.035
permiso_exportador	4500	0.062
lote_uva	1785	0.058
analisis_calidad	1785	0.024
ingreso	2500	0.048
renglon_ingreso	7522	0.122
transferencia	2500	0.025
renglon_transferencia	10005	0.158
comprobante	150000	2.592
renglon_comprobante	600575	14.656
stock_deposito	400	0.007
stock_sala	5426	0.084
auditoria_precio	15	-

Concluida la carga masiva (y tras rehabilitar los triggers), se ejecutó el script validacion_carga.sql para verificar la integridad referencial y de dominio de los datos históricos insertados. Los resultados confirmaron un estado 100% íntegro de la base de datos:

- 0 filas huérfanas o con dependencias rotas.
- 0 casos de stock negativo.
- 0 comprobantes con estados o datos inconsistentes.
- 0 vinos cuya composición porcentual difiera del 100%.
- 0 permisos de exportador inválidos o fuera de término.
- 0 clientes con aplicación de descuentos inconsistentes con su categoría.
- 0 lotes de uva duplicados.

5. Índices y rendimiento:

5.1. Tabla comparativa antes y después:

Consulta	Estado	Tipo de Scan Predominante	Filas Leídas / Descartadas	Buffers (Hit)	Planning Time	Execution Time
S4	ANTES	Seq Scan en renglon_transferencia + Seq Scan transf.	22 dev. (9925 desc.) / 200 (2300)	hit: 68	0.246 ms	0.845 ms
S4	DESPUÉS	Bitmap Index Scan (idx_fk_renglon_transf_vino + fecha)	22 dev. (0 desc.) / 200 (0 desc.)	hit: 35 (-48.5%)	0.259 ms	0.182 ms (-78%)
S5	ANTES	Seq Scan en transferencia	0 devueltas (2500 descartadas)	hit: 15	0.132 ms	0.128 ms
S5	DESPUÉS	Index Only Scan (idx_transferencia_pendientes)	0 devueltas (0 descartadas)	hit: 2 (-86.7%)	0.153 ms	0.059 ms (-54%)
S8	ANTES	Parallel Seq Scan en comprobante (2 workers)	32564 dev. (117436 descartadas)	hit: 1516	0.126 ms	13.341 ms
S8	DESPUÉS	Index Only Scan (idx_comprobante_fecha_sala_inc)	32564 dev. (0 descartadas)	hit: 191 (-87.4%)	0.138 ms	8.614 ms (-35%)
S3	ANTES	Parallel Seq Scan en comprobante (2 workers)	507 dev. (149492 descartadas)	hit: 3632	0.461 ms	10.084 ms
S3	DESPUÉS	Index Only Scan (idx_comprobante_fecha_sala_inc)	507 dev. (0 descartadas)	hit: 1157 (-68%)	0.502 ms	3.119 ms (-69%)

5.2. Planes de ejecución:

El análisis de los planes de ejecución con EXPLAIN ANALYZE demostró cambios significativos:

1. Transición de recorridos secuenciales a escaneos de índice (S4):
Originalmente, el motor resolvía la consulta mediante un Seq Scan sobre las tablas renglon_transferencia y transferencia, descartando 9.925 filas en el proceso. Tras la creación de los índices (idx_fk_renglon_transf_vino y idx_transferencia_fecha_sala), el planificador cambió a un Bitmap Index Scan, lo que le permitió no descartar ninguna fila durante la lectura y reducir el acceso a memoria (Buffers hit) casi a la mitad, pasando de 68 a 35 hits.

2. Optimización de agrupaciones y lecturas exclusivas en índice (S8)
Antes de la optimización, la consulta S8 forzaba al motor a realizar un Parallel Seq Scan utilizando 2 workers para barrer la tabla comprobante, descartando 117.436 filas y generando un costo de lectura de 1.516 buffers hit.
Al crear el índice compuesto con columnas incluidas (idx_comprobante_fecha_sala_inc), el planificador reconoció que los datos necesarios (total e id_cliente) estaban en la propia estructura del árbol. Por consiguiente, el plan de ejecución migró a un Index Only Scan directo, eliminando los descartes de filas y reduciendo el consumo de memoria interna en un 87,4% (bajando a 191 hits).

3. Caso de índice ignorado por el planificador (S1)
Para la consulta S1, se creó inicialmente un índice sobre codigo_vino en la tabla RENGLON_COMPROBANTE. Sin embargo, el planificador decidió ignorarlo, manteniendo la estrategia de Parallel Seq Scan sobre el 100% de la tabla (aprox. 600.000 filas).

5.3. Costo de escritura:

Para evaluar el impacto de las estructuras de índices B-Tree en las operaciones de escritura, se realizó una prueba de inserción masiva sobre la tabla RENGLON_COMPROBANTE. Los tiempos de ejecución obtenidos fueron los siguientes:

- **Inserción masiva SIN índices:** 1.210,316 ms.
- **Inserción masiva CON índices:** 4.167,916 ms.

Estos resultados evidencian un incremento del 244,4% en el tiempo de procesamiento, volviendo la operación 3,44 veces más lenta. Esta penalización refleja el costo operativo del motor de base de datos, el cual debe actualizar los árboles de búsqueda por cada fila insertada. Este alto impacto en las inserciones fue el factor decisivo para descartar índices (como el evaluado para la consulta S1) que no demostraron una mejora real en los tiempos de lectura, evitando así penalizar innecesariamente el rendimiento del sistema.

6. Consultas SQL:

Destacamos las siguientes resoluciones por su complejidad estructural o interés personal:

- **S10 - Rendimiento en kg/ hectárea y subconsulta correlacionada:** Resuelve la comparación de rendimientos calculando primero la métrica individual mediante una expresión de tabla común (CTE). Luego, aplica una subconsulta correlacionada en la cláusula WHERE para filtrar las parcelas que rinden por debajo del promedio específico de su propia finca en esa campaña.
- **S11 - Funciones de ventana:** Para obtener el ranking de ventas y la facturación acumulada mensual por sala, se implementó la cláusula RANK() OVER (PARTITION BY id_sala, mes) para establecer el podio de los más vendidos, combinada con una suma acumulativa SUM(ingreso_vino) OVER (PARTITION BY id_sala ORDER BY mes).
- **S12 - División relacional:** Responde a la búsqueda de clientes que compraron la totalidad de los vinos de una bodega aplicando una doble negación lógica (NOT EXISTS anidados), simulando de manera estricta la operación de división del álgebra relacional.
- **I3 e I6 - Inserción simultánea de cabecera y detalle:** Emplean una CTE combinada con la cláusula RETURNING para registrar un nuevo documento, capturar de forma instantánea su ID autogenerado, e inyectarlo inmediatamente en la inserción de los renglones correspondientes dentro de una única transacción.
- **D5 - Choque con triggers:** Se documentó un caso de bloqueo por integridad. Al intentar eliminar un renglón de ingreso cuyo estado sea "procesado", la operación queda anulada por la ejecución automática del trigger trg_renglon_ingreso_bloqueo, disparado en el evento BEFORE DELETE.

7. API y front:

7.1. Arquitectura:

La API fue construida utilizando **FastAPI** bajo una arquitectura asincrónica (async/await) en Python. Esta elección técnica garantiza un alto rendimiento en las operaciones de I/O de la base de datos mediante el uso de un connection *pool* asincrónico gestionado en el lifespan de la aplicación.

Se implementó el patrón de **Middlewares** y manejo de contextos para dos requerimientos fundamentales:

1. **CORS:** Habilitación de peticiones cruzadas para permitir el consumo desde el Frontend.
2. **Auditoría y Transacciones:** Se diseñó un middleware a nivel HTTP (set_app_user_middleware) que intercepta el token de autorización (Bearer token-{id}:{email}), extrae la identidad del usuario y la inyecta en el contexto de la transacción (app_current_user). Esto permite que los triggers de auditoría en la base de datos (por ejemplo, el de variación de precios) registren automáticamente el autor del cambio sin necesidad de alterar las firmas de las funciones SQL.

7.2. Endpoints:

El sistema expone las diferentes entidades y operaciones a través de un ruteador organizado por dominios. A continuación, se destacan los endpoints principales evaluados en los test de integración:

- **Consultas y Reportes (GET):**
    - /api/vinos y /api/clientes: Listados paginados.
    - /api/reportes/top-vinos: Ranking de los vinos más vendidos (Consulta S1).
    - /api/reportes/ventas-por-sala: Facturación acumulada trimestral (Consulta S8).
    - /api/reportes/trazabilidad/{cod}: Composición, cepas y maridajes (Consulta S9).
- **Gestión Transaccional (POST):**
    - /api/comprobantes, /api/transferencias, /api/ingresos: Creación de documentos comerciales y movimientos de stock en estado inicial ("en confección").
    - /api/transferencias/{id}/procesar: Endpoint de cambio de estado.

7.3. Traducción de errores:

Para evitar la exposición de detalles internos de la base de datos (como el nombre de las tablas o las sentencias SQL) al cliente, se desarrolló un global_exception_handler. Este handler intercepta las excepciones de psycopg y traduce los códigos de estado internos (SQLSTATE) a códigos de error HTTP estandarizados:

- **Violaciones de Unicidad (23505):** Traducidas a 409 Conflict (ej. intento de crear un vino duplicado).
- **Violación de Clave Foránea (23503):** Traducidas a 409 Conflict (ej. intento de crear un cliente en un código postal inexistente).
- **Violación de CHECK (23514) y NOT NULL (23502):** Traducidas a 422 Unprocessable Entity (ej. graduación alcohólica fuera de rango).
- **Reglas de Negocio / Triggers (P0001):** Las excepciones lanzadas manualmente desde los triggers (con RAISE EXCEPTION) son mapeadas a 409 Conflict, devolviendo mensajes claros como "Stock insuficiente" (RN-09).
- **Errores de Transacción (40001 Serialization, 40P01 Deadlock):** Traducidas a 503 Service Unavailable.

7.4. Experiencias con el Frontend:

El armado del frontend fue un desafio nuevo para nosotros, nunca habíamos hecho antes un frontend funcional para una aplicación completa. Empezamos armando un prototipo visual no funcional en figma, ahí pudimos darle algo de diseño e implementar las pantallas que íbamos a necesitar. Una vez que el diseño estuvo terminado, usamos un plugin de figma que nos permitió exportar y traducir ese diseño visual a una estructura de código inicial en React.

Para el momento de hacer que este frontend utilizable utilizamos IA, de esta forma sin tener mucha experiencia nos las arreglamos para lograr estructurar la navegación del sistema y vincular los componentes del frontend con la API del servidor. Cuando la aplicación ya estuvo funcionando y operando correctamente de forma local aprovechamos lo visto en la materia de Desarrollo de Software para crear un contenedor en Docker que nos permitiera correr la aplicación en otros entornos.

8. Conclusiones:

Creemos que desarrollar este proyecto fue posible en estas semanas gracias a que había mucho detalle y especificaciones en las consignas que estaban en el trabajo practico, fue una guía bastante útil porque nos permitió saber cuándo una decisión tenía sentido y más o menos por donde encaminar las cosas desde un inicio. En algunas etapas sentimos que nos llevaba demasiado de la mano, pero para las ocasiones en las que no sabíamos como seguir fue un buen salvavidas.

Dejando eso de lado también estuvimos bastante contentos con desarrollar el trabajo porque pudimos poner a prueba lo que habíamos aprendido y ver realmente el uso que se le suelen dar a los temas que vimos en el año. Esto sentimos que no solo aplica a Base de Datos sino también a Desarrollo de Software, que si bien tiene su proyecto a parte no esta tan centrado en el uso de la base de datos, entonces ver como combinamos lo que sabíamos de cada materia para armar este proyecto fue bastante motivador.

Mas allá de las dificultades y problemas inesperados que encontramos en el medio del desarrollo, todos los integrantes quedamos muy contentos con el resultado final. Creemos que el esfuerzo valió la pena y que la experiencia práctica fue muy entretenida.